발표 대본

LangChain 버전 마이그레이션 코드 어시스턴트 · SKN35 3rd 4Team · 15 분 (본편 약 14 분 20 초 + 질문) · 발표 [이름]

화면은 훅만	장마다 세 박자	멈추기
숫자 · 한 문장 · 그림을 가리키고, 설명은 말로	훅 한 마디 → 설명 → 넘기는 말로 다음 장 예고	숫자를 말한 뒤, 질문을 던진 뒤 1 초 쉬기

01 표지 0:00 ~ 0:30

훅 LLM 이 준 코드, 그대로 믿어도 될까요?

안녕하세요, 4 팀 발표를 맡은 [이름]입니다.

저희는 옛 버전 LangChain 코드를 넣으면, 원하는 버전에서 돌아가는 코드로 고쳐 주고 정말 맞는지 검사까지 하는 챗봇을 만들었습니다.

오늘 핵심 단어는 두 개, 근거와 검증입니다.

→ **넘기며** 먼저 순서를 말씀드리겠습니다.

02 목차 0:30 ~ 0:50

훅 네 부분으로 말씀드리겠습니다.

어떤 문제를 풀었는지, 팀이 무엇을 만들었는지, 어떻게 평가했고 결과가 어땠는지, 마지막으로 만들면서 배운 것입니다.

→ **넘기며** 먼저 문제입니다.

03 에러 코드 0:50 ~ 1:40

훅 첫 줄부터 에러입니다.

ChatGPT 에게 LangChain 코드를 부탁하면 이런 코드를 줍니다.

최신 LangChain 에서 실행하면 첫 줄 import 부터 에러가 납니다.

1.0 에서 패키지가 나뉘고, LLMChain 같은 옛 체인이 빠졌기 때문입니다.

더 큰 문제는 LLM 이 이 답이 틀렸다는 걸 모른다는 점입니다.

→ **넘기며** 왜 이런 일이 생길까요?

04 숫자 세 개 1:40 ~ 2:30

훅 문서를 줘도 66%입니다.

인터넷 코드는 대부분 옛 버전이라, LLM 은 옛 방식을 답합니다. 이걸 버전 지체 환각이라고 부릅니다.

연구에서도 문서가 부족하면 42%, 구조화된 문서를 줘도 66%만 목표 버전에서 실행됐습니다.

문서를 주는 것만으로는 부족하고, 고친 코드를 확인하는 단계가 필요하다는 뜻입니다.

→ **넘기며** 그래서 저희는 두 가지를 더했습니다.

05 비교 표 2:30 ~ 3:20

훅 근거로 고치고, 검사로 한 번 더 확인합니다.

첫째, 기억이 아니라 대응표와 최신 공식 문서를 근거로 고칩니다.

둘째, 고친 코드를 LLM 없이 11 가지로 검사하고, 틀리면 이유를 들고 다시 고칩니다. 최대 세 번입니다.

버전도 사용자가 질문에 적은 대로 맞춥니다. '0.2 로 되돌려 줘'라고 하면 0.2 로, 적지 않으면 최신으로 바꿉니다.

→ **넘기며** 이제 팀이 무엇을 만들었는지 보겠습니다. 먼저 근거입니다.

06 데이터 숫자 3:20 ~ 4:10

훅 근거 9,155 줄을 미리 만들어 두었습니다.

2 번이 만든 근거입니다. 옛 import 경로를 새 경로로 잇는 대응표가 9,155 줄, 옛 메서드를 잇는 대응표가 42 개입니다.

공식 문서 295 개를 11,945 개 조각으로 나눠 검색할 수 있게 했고, 버전마다 실제로 있는 기능 목록도 42 개 만들었습니다.

그래서 최신으로 올리는 것뿐 아니라 옛 버전으로 되돌릴 때도 근거를 찾을 수 있습니다.

→ **넘기며** 이 근거를 어떻게 쓰는지, 흐름을 보겠습니다.

07 흐름 그림 4:10 ~ 5:20

훅 검사에 실패하면, 이유를 들고 다시 찾습니다.

질문이 들어오면 1 번이 만든 최상위 그래프가 먼저 질문을 분류합니다. 코드 변환, LangChain 질문, 일반 질문 세 갈래입니다.

코드 변환이면 네 단계를 거칩니다. 바꿀 부분을 찾고, 근거를 찾고, 근거로 고치고, 검사합니다.

검사에 실패하면 실패 이유를 들고 다시 근거를 찾습니다. 최대 세 번입니다.

LangChain 질문은 같은 문서를 검색해서 답하고, 일반 질문은 바로 답합니다.

→ **넘기며** 핵심인 검사를 조금 더 보겠습니다.

08 검사 11 가지 5:20 ~ 6:10

훅 검사에는 LLM 을 쓰지 않습니다.

3 번이 만든 검사는 11 가지인데, LLM 을 쓰지 않습니다. 검사하는 LLM 도 같은 옛 지식을 갖고 있기 때문입니다.

새 경로가 정말 있는지, 근거 없이 바뀐 줄은 없는지, 결과 모양을 잃지 않았는지를 코드로 확인합니다.

근거 없이 바뀐 줄은 곧 환각이라서, 바로 걸러 냅니다.

→ **넘기며** 이 결과를 사용자는 어떻게 볼까요?

09 화면 6:10 ~ 7:10

훅 기다리는 동안 무엇을 하는지 보입니다.

제가 맡은 화면입니다. 코드를 넣고 말로 부탁하면 됩니다.

기다리는 동안 지금 어느 단계인지, 어떤 근거를 찾았고 어느 줄을 고쳤는지, 검사에서 떨어져 다시 고치는 중인지가 실시간으로 보이고, 끝나면 그 과정이 답 위에 남습니다.

결과가 나오면 지운 줄은 빨강, 새 줄은 초록으로 보이고, 변경 사항, 근거, 11 가지 검사 결과를 탭으로 볼 수 있습니다.

바뀐 코드와 변경 리포트는 파일로 내려받을 수 있고, 실패하면 버튼 하나로 다시 시도할 수 있습니다.

같은 대화 안에서는 앞에서 한 말을 기억해서, 이어서 물어봐도 맥락을 이어 갑니다.

→ **넘기며** 대화는 어디에 남을까요?

10 ERD 7:10 ~ 7:50

훅 대화는 TiDB 에 남습니다.

팀에서 정한 대로 테이블은 두 개입니다. 사용자, 그리고 질문과 답 한 쌍이 한 행인 대화 기록입니다.

새 대화를 누르면 세션이 새로 생기고, 가입할 때 아이디 중복을 검사합니다.

DB 에 연결할 수 없어도 화면은 멈추지 않고 로컬에 저장하면서 이유를 알려 줍니다.

→ **넘기며** 이제 어떻게 평가했는지 말씀드리겠습니다.

11 실험 설계 7:50 ~ 8:40

훅 같은 질문 30 개, 같은 조건으로 비교합니다.

실제 사용자처럼 요청 문장과 옛 코드로 질문 30 개를 만들었습니다. 열 가지 유형을 세 개씩 담았습니다.

모두 최신 버전에서 실제로 깨지는 코드인지 미리 확인했습니다.

일반 LLM 과 같은 gpt-4o-mini 모델, 같은 질문으로 비교했습니다. 채점은 사람이나 LLM 이 아니라 코드로 했습니다.

옛 import 가 남았는지, 새 import 가 1.x 에 진짜 있는지, 결과값이 그대로인지 등 공통 기준 여덟 가지를 두 답에 똑같이 적용했습니다.

→ **넘기며** 실제로 돌려 본 결과부터 보여 드리겠습니다.

12 실제 실행 (10-14) 8:40 ~ 9:40

훅 처음 두 번은 떨어지고, 세 번째에 통과했습니다.

왼쪽은 예전 방식의 LLMChain 코드입니다. 오른쪽처럼 최신 방식으로 바뀌었습니다.

처음 두 번은 검사에 걸렸고, 실패 이유를 들고 다시 고쳐서 세 번째에 통과했습니다.

바뀐 여섯 줄 모두 근거가 붙었습니다.

→ **넘기며** 오늘 아침에는 화면 전체를 최상위 그래프에 연결했습니다.

13 세 갈래 연결 (10-15) 9:40 ~ 10:40

훅 질문 하나로, 알아서 세 갈래로 답합니다.

인사를 하면 2 초 만에 일반 답을, LangChain 변화를 물으면 문서를 찾아 12 초 만에 답합니다.

코드를 주고 최신으로 바꿔 달라고 하면, 질문에서 목표 버전 1.4.5 를 읽어 16 초 만에 한 번에 통과했습니다.

되돌리기 요청도 목표 버전은 정확히 읽었지만, 아직 검사를 통과하지 못하는 경우가 있습니다. 이 부분은 뒤에서 말씀드리겠습니다.

→ **넘기며** 그럼 일반 LLM 보다 얼마나 나을까요?

14 결과 숫자 · 막대그래프 10:40 ~ 11:20

훅 같은 모델인데, 57% 대 0%입니다.

1.x 로 올리는 질문 30 개에서, 공통 기준 여덟 가지를 모두 통과한 답은 저희 챗봇 57%, 일반 LLM 0%였습니다. (1 초 쉬기)

일반 LLM 은 30 개 중 29 개에서 옛 import 를 그대로 두고, '1.x 에 맞게 수정했다'고 답했습니다.

앞에서 말씀드린 버전 지체 환각이 저희 실험에서도 그대로 나왔습니다.

정답 import 항목을 빼고 '실행은 되는 코드인가'만 보면 87% 대 0%입니다. 모델을 바꾼 게 아니라, 근거와 검사를 붙인 차이입니다.

→ **넘기며** 다시 고치는 과정은 얼마나 효과가 있었을까요?

15 재시도 효과 · 세로 막대 11:20 ~ 11:50

훅 다시 고쳐서 10%p, 그리고 실패는 숨기지 않습니다.

첫 시도에서 검사를 통과한 건 43%, 실패 이유를 들고 다시 고치면 53%까지 올라갔습니다.

통과하지 못한 답은 화면에 '검증 실패'와 이유를 그대로 보여 줍니다. 일반 LLM 은 틀려도 항상 '수정했다'고 말합니다.

바뀐 줄의 93%에는 근거가 붙어 있어서, 사용자가 직접 확인할 수 있습니다.

→ **넘기며** 실험은 약점도 정확히 보여 줬습니다.

16 약점 네 가지 11:50 ~ 12:50

훅 약점도 숫자로 찾았고, 고칠 곳도 정했습니다.

첫째, 고쳐 달라면서 에러 이유도 같이 물으면 설명만 하고 끝나는 경우가 30 개 중 6 개였습니다. 분류 규칙을 보강하면 됩니다.

둘째, 실패 원인 1 위는 기록 없이 줄을 바꾸는 환각이었습니다. 다만 검사기가 이걸 막아서, 환각이 '통과'로 나간 적은 없습니다.

셋째, 검사기가 놓친 경우도 찾았습니다. 결과를 ['text']로 잘못 꺼내는 코드는 평가 직후 3 번이 바로 고쳤습니다. 옛 버전으로 되돌릴 때도 1.x 기준으로 검사하는 부분은 남아 있습니다. 버전별 도구 목록은 이미 있어서 연결만 하면 됩니다.

넷째, 최신 방식 대신 옛 창고(classic)로 우회한 답이 세 개 있었습니다.

→ **넘기며** 그래서 저희가 정한 규칙으로 스스로도 채점해 봤습니다.

17 자체 평가 표 12:50 ~ 13:40

훅 자체 평가 83 점, 약한 곳부터 고칩니다.

팀이 프롬프트와 코드에 적어 둔 규칙 21 가지를 실제 실행 기록으로 점검했고, 100 점 만점에 83 점이 나왔습니다.

분석, 근거 검색, 답변 규칙, 안정성은 목표를 넘었습니다. 근거 검색은 대응표에 있는 옛 import 를 99% 찾아왔습니다.

약한 곳은 질문 분류, 환각, 검증 루프입니다. 고칠 순서도 정했습니다. 분류 규칙, 없던 줄 금지와 틀린 근거 빼기, 목표 버전별 검사 순입니다.

한계도 있습니다. 질문마다 한 번씩 돌렸고, 코드를 실제로 실행하지는 않았습니다.

→ **넘기며** 마지막으로 정리하겠습니다.

18 가치 13:40 ~ 14:20

훅 라이브러리가 바뀔 때마다, 근거와 함께 확인된 코드.

에러를 하나씩 검색하는 대신, 근거가 붙은 답을 바로 확인할 수 있습니다.

같은 모델로 0%였던 것을 57%로 올렸고, 실패하면 실패라고 알려 줍니다. 그래서 믿고 쓸 수 있습니다.

대응표와 문서, 검사 기준만 바꾸면 다른 라이브러리에도 적용할 수 있습니다.

→ **넘기며** 함께 만든 팀을 소개하겠습니다.

19 팀 · 감사 14:20 ~ 15:00

훅 감사합니다. 질문 받겠습니다.

1 번이 그래프와 질문 분류, 2 번이 데이터와 검색, 3 번이 변환과 검사, 4 번인 제가 화면과 DB, 평가를 맡았습니다.

각자 막혔던 곳도 있었습니다. 노드끼리 주고받는 형식과 분기 이름을 맞추고, 133MB 벡터 DB 를 클라우드로 옮기고, 검사기가 속지 않게 규칙을 늘리고, 배포 키 문제와 공정한 채점을 풀었습니다. 전체 기록은 최종 보고서에 정리했습니다.

감사합니다. 질문 받겠습니다.

20 부록 1 · 파이프라인 질문 시

훅 (설계 질문이 나오면) 파이프라인을 한 장으로 보여 드리겠습니다.

위는 미리 한 번 만드는 근거입니다. 패키지 코드와 공식 변환 규칙으로 대응표 9,155 줄, 문서 295 개를 11,945 개 조각으로 나눠 Chroma Cloud 에 넣었습니다.

아래는 질문 하나가 지나가는 길입니다. 분류, 바꿀 부분 찾기, 근거 검색, 근거를 붙여 다시 쓰기, 그리고 LLM 없는 검사입니다.

검색은 대응표 정확 매칭을 먼저 하고, 못 찾으면 BM25 와 벡터 검색을 합칩니다. 모든 단계는 State 하나로 이어집니다.

21 부록 2 · 파일 구조 질문 시

훅 (코드 위치 질문이 나오면) 파일은 역할별로 나뉘어 있습니다.

그래프는 src/graph, 검색과 데이터는 src/rag 와 data, 검사는 verifier_node, 화면은 streamlit_app.py 와 src/ui, 평가는 eval 폴더입니다.

22 부록 3 · 트러블슈팅 질문 시

훅 (어려웠던 점 질문이 나오면) 역할별로 막혔던 것과 푼 방법입니다.

가장 컸던 건 노드끼리 형식이 달라 전체 실행이 안 되던 문제였고, State 와 분기 이름을 통일해 풀었습니다.

검사기는 LLM 이 옛 근거 ID 를 복사하는 걸 막기 위해 '이번에 보여 준 근거만' 인정하도록 바꿨습니다.

배포에서는 키를 읽는 순서 때문에 그래프가 연결되지 않았는데, 키를 먼저 읽고 실패는 기억하지 않게 고쳤습니다.

예상 질문과 답

Q. 통과하면 정답인가요?

A. 아닙니다. 코드 검사까지이고, 실행 결과 비교는 다음 단계입니다. (17 장)

Q. 왜 검사를 LLM 에게 맡기지 않았나요?

A. 검사하는 LLM 도 같은 옛 지식을 갖고 있어서, 코드로 확인했습니다. (8 장)

Q. 근거를 지어낼 수 있지 않나요?

A. 이번에 실제로 보여 준 근거 ID 만 인정하고, 기록 없이 바뀐 줄은 환각으로 걸러 냅니다. 오늘 실제로 잡아낸 사례가 있습니다. (16 장)

Q. 버전은 어떻게 고르나요?

A. 질문에 적으면 됩니다. 그래프가 글에서 목표 버전을 읽고, 없으면 최신 1.4.5 로 바꿉니다.

Q. 왜 1 분 가까이 걸리나요?

A. 검사에 실패하면 최대 세 번 다시 고치기 때문입니다. 화면은 기다리는 동안 단계를 실시간으로 보여 줍니다.

Q. 대화는 어디에 저장되나요?

A. TiDB 에 사용자 · 대화 두 테이블로 저장합니다. (10 장)

Q. 30 개로 충분한가요?

A. 열 가지 유형 · 난이도별 경향을 보는 게 목적입니다. 다음엔 실사용 코드로 늘립니다.

Q. 가장 어려웠던 점은요?

A. 노드끼리 주고받는 형식과 분기 이름이 달라 전체 실행이 안 되던 것입니다. State 와 이름을 통일해 풀었습니다. (부록 3)

Q. 검색은 어떻게 하나요?

A. 대응표 정확 매칭을 먼저 하고, 못 찾으면 BM25 와 벡터 검색을 RRF 로 합쳐 상위 5 개를 씁니다. 목표 버전별로 경로도 판정합니다. (부록 1)

Q. 57%면 낮은 것 아닌가요?

A. 같은 모델이 혼자서는 0%였습니다. 그리고 통과 못 한 답은 '검증 실패'로 알려 줘서 사용자가 그대로 믿지 않게 합니다. 실패 원인을 숫자로 찾았고, 고칠 순서도 정했습니다. (16 · 17 장)

Q. 채점이 공정한가요?

A. LLM 이나 사람이 아니라 코드로 채점했습니다. 두 답에 같은 여덟 가지를 똑같이 적용했고, 몇 번을 채점해도 같은 점수가 나옵니다.

Q. 버전 되돌리기는요?

A. 목표 버전은 정확히 읽지만, 검사기가 아직 1.x 기준이라 6 개 중 정답을 맞춘 답이 없었습니다. 2 번의 버전별 도구 목록을 검사기에 연결하는 게 다음 단계입니다.

[ ] 칸은 실험 결과 · 이름이 정해지면 채우기